

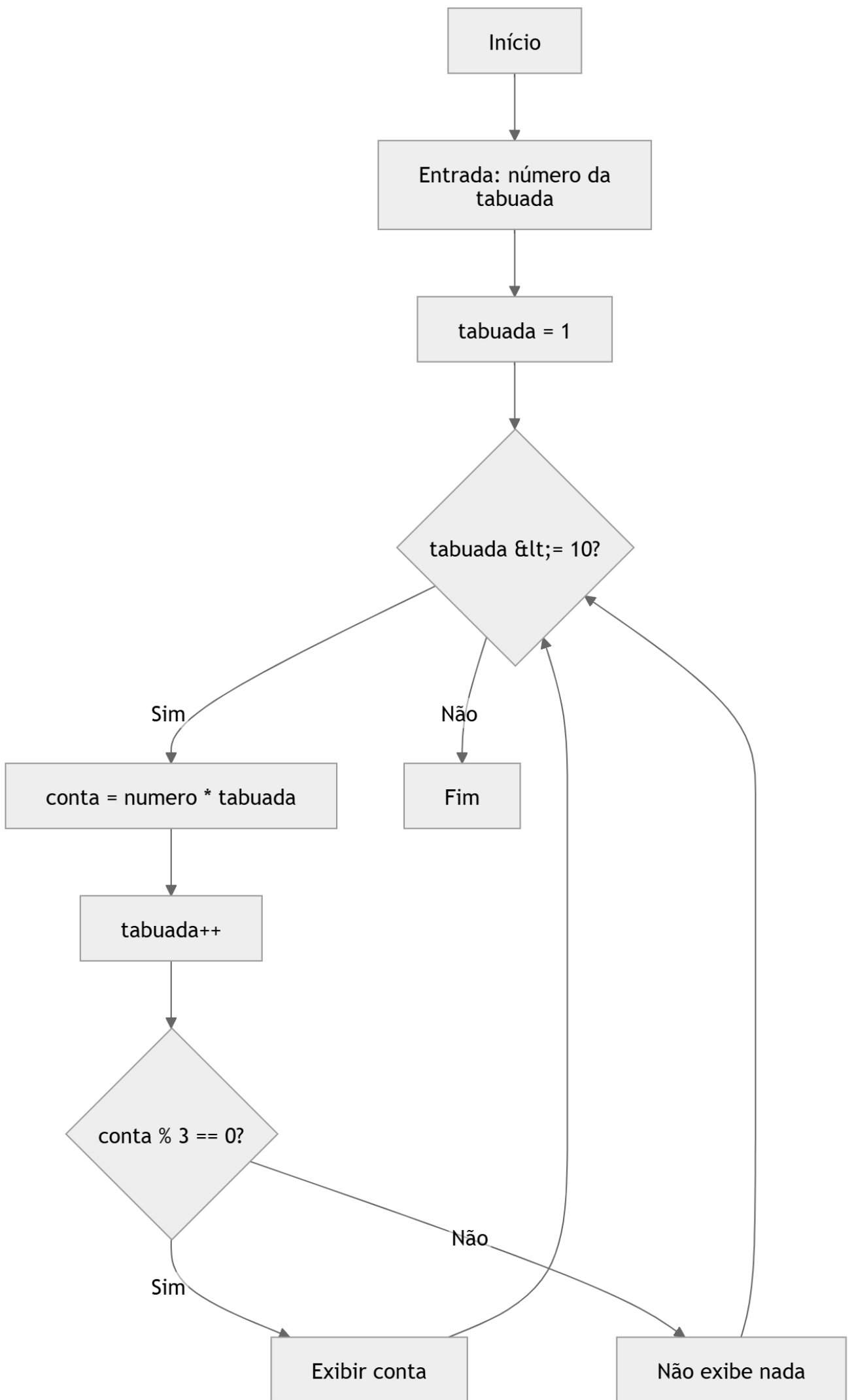
Algoritmos e Programação – AP

▪ Exercício Proposto 1

- **Nível:** Fácil
- **Enunciado:** Solicite um número inteiro ao usuário e imprima a sua tabuada de 1 a 10, mas apenas os resultados que forem múltiplos de 3.

```
import java.util.Scanner;

public class multiplos {
    Run | Debug
    public static void main(String[] args){
        Scanner scanner = new Scanner(System.in);
        System.out.println(x: "Digite um numero para ver sua tabuada");
        int numero = scanner.nextInt();
        int tabuada = 1;
        int conta;
        while(tabuada <= 10){
            conta=numero*tabuada;
            tabuada++;
            if(conta%3==0){
                System.out.println(conta);
            }
        }
        scanner.close();
    }
}
```



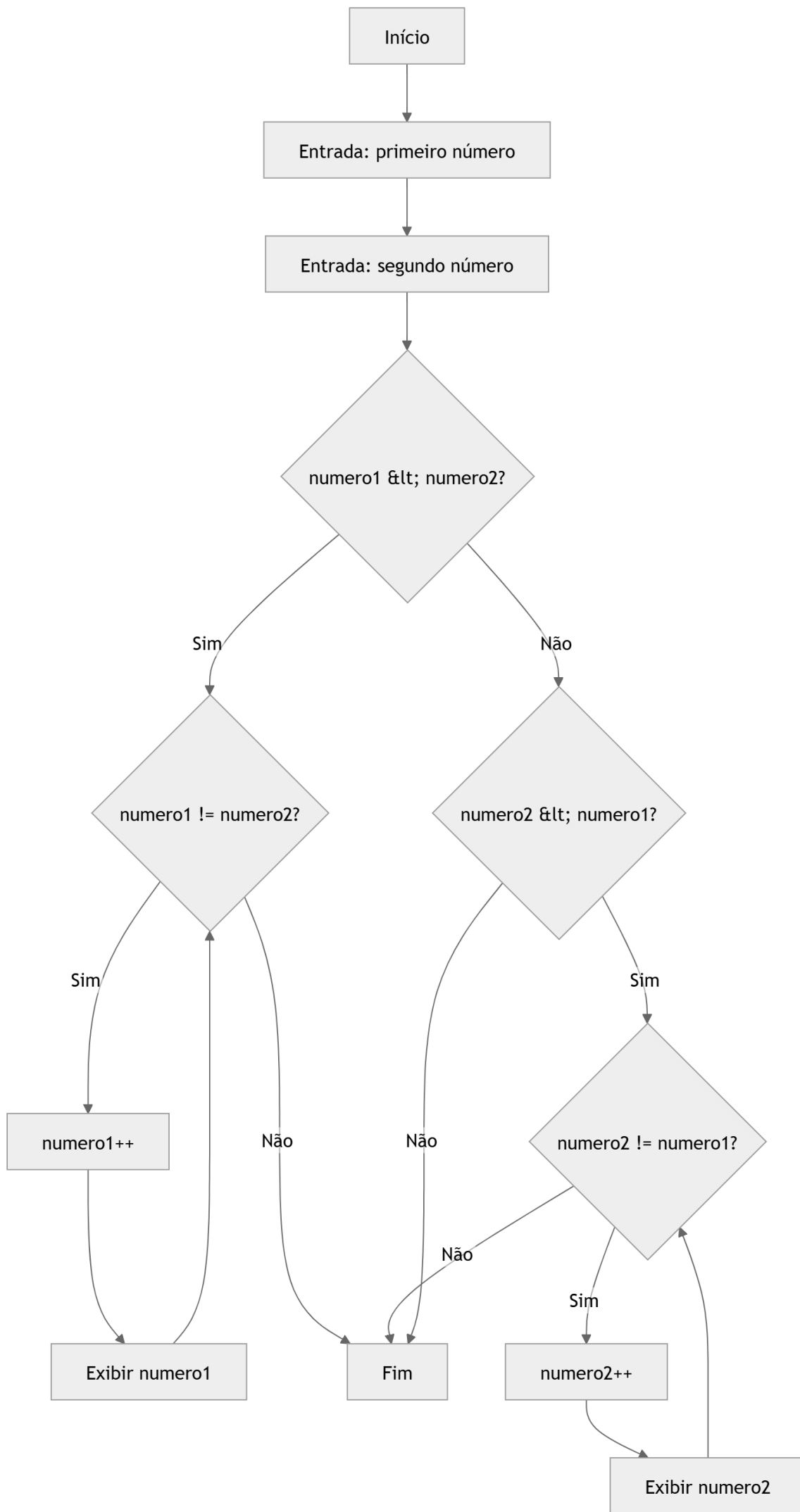
Algoritmos e Programação – AP

▪ Exercício Proposto 2

- **Nível:** Fácil
- **Enunciado:** Leia dois números inteiros (a e b). Utilizando obrigatoriamente o comando *while*, imprima todos os números inteiros existentes entre eles em ordem crescente.

```
import java.util.Scanner;
public class numero{
    Run | Debug
    public static void main(String[] args){
        Scanner scanner = new Scanner(System.in);
        System.out.println(x: "Coloque o primeiro numero");
        int numero1 = scanner.nextInt();
        System.out.println(x: "Coloque o segundo numero");
        int numero2 = scanner.nextInt();
        if(numero1<numero2){
            while(numero1 != numero2){
                numero1++;
                System.out.println(numero1);
            }
        }
        if(numero2<numero1){
            while(numero2 != numero1){
                numero2++;
                System.out.println(numero2);
            }
        }

        scanner.close();
    }
}
```



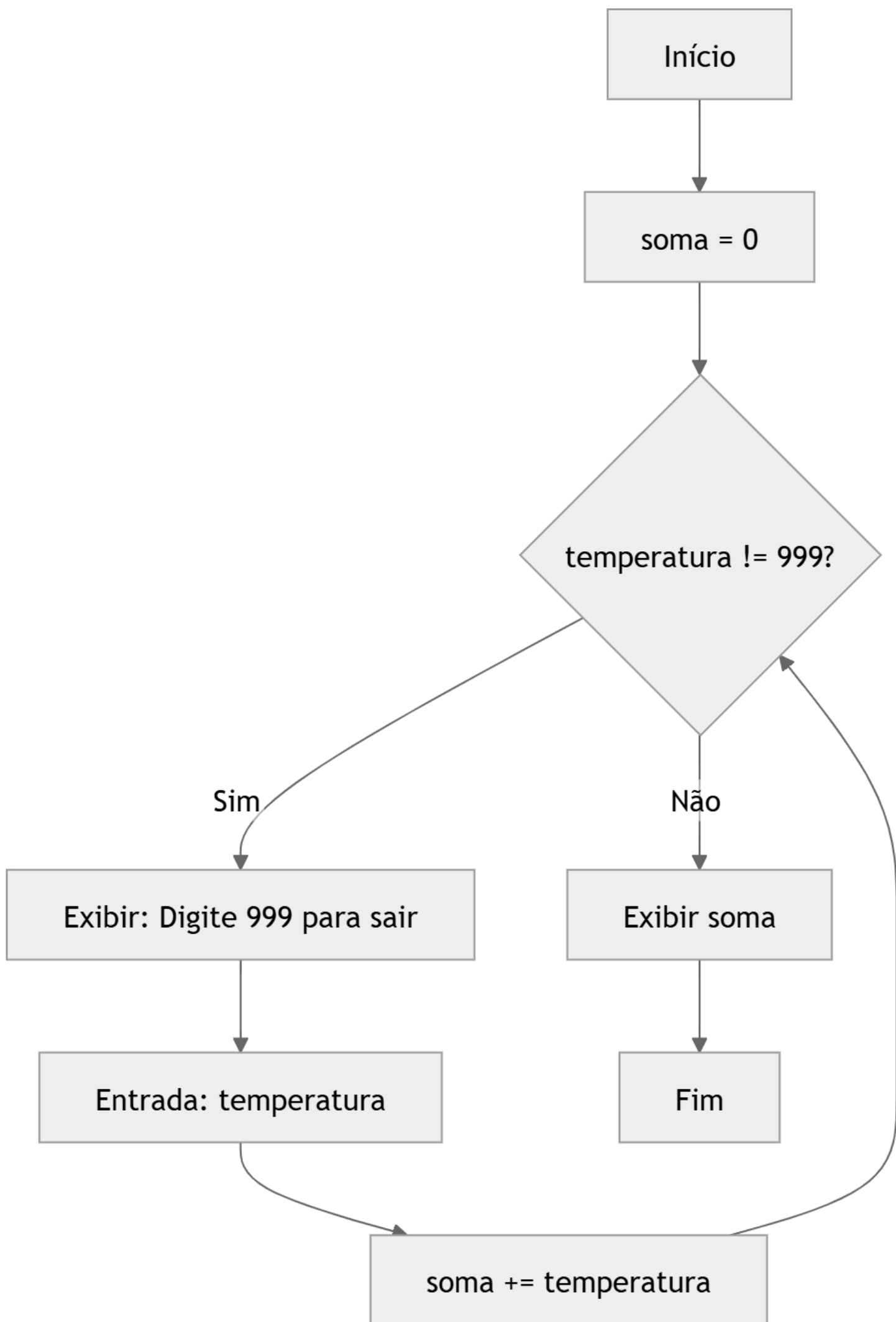
Algoritmos e Programação – AP

▪ Exercício Proposto 3

- **Nível:** Fácil
- **Enunciado:** Um laboratório de solos precisa ler diversas medições de temperatura. O programa deve ler temperaturas até que o usuário digite o valor sentinela 999. Ao final, exiba a média aritmética das temperaturas válidas lidas.

```
import java.util.Scanner;
public class laboratorio {
    Run | Debug
    public static void main(String[] args){
        Scanner sc= new Scanner(System.in);
        double temperatura = 0;
        double soma = 0;
        while(temperatura!=999){
            System.out.println(x: "Digite 999 para sair");
            temperatura = sc.nextInt();
            soma+=temperatura;
        }
        System.out.println(soma);

        sc.close();
    }
}
```

Algoritmos e Programação – AP

▪ Exercício Proposto 4

- **Nível:** Fácil
- **Enunciado:** Crie um programa que receba um número inteiro positivo e realize uma contagem regressiva até *zero*. A cada iteração, o programa deve exibir a mensagem:

*"Sistema operando. T-menos [**valor**] segundos;"*

```
import java.util.Scanner;

public class relógio{
    Run | Debug
    public static void main(String[] args){
        Scanner scanner = new Scanner(System.in);
        System.out.println(x: "Digite um numero");
        int numero = scanner.nextInt();
        while(numero != 0){
            numero--;
            System.out.println("Sistema operando. T-menos "+numero+" segundos");
        }
        System.out.println(x: "Você chegou no final");

        scanner.close();
    }
}
```

Início

Entrada: digitar número

numero != 0?

Sim

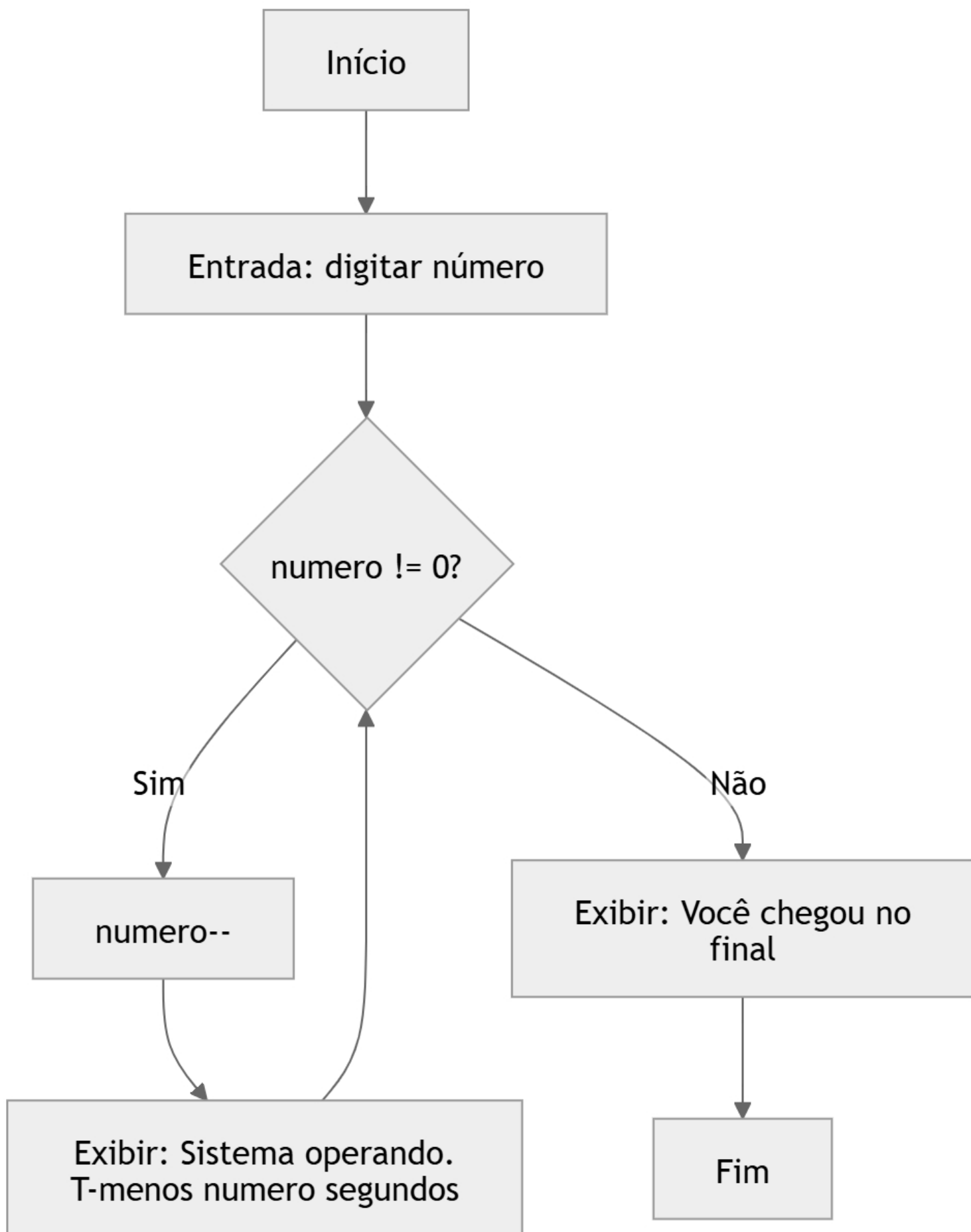
numero--

Não

Exibir: Você chegou no
final

Exibir: Sistema operando.
T-menos numero segundos

Fim



Algoritmos e Programação – AP

▪ Exercício Proposto 5

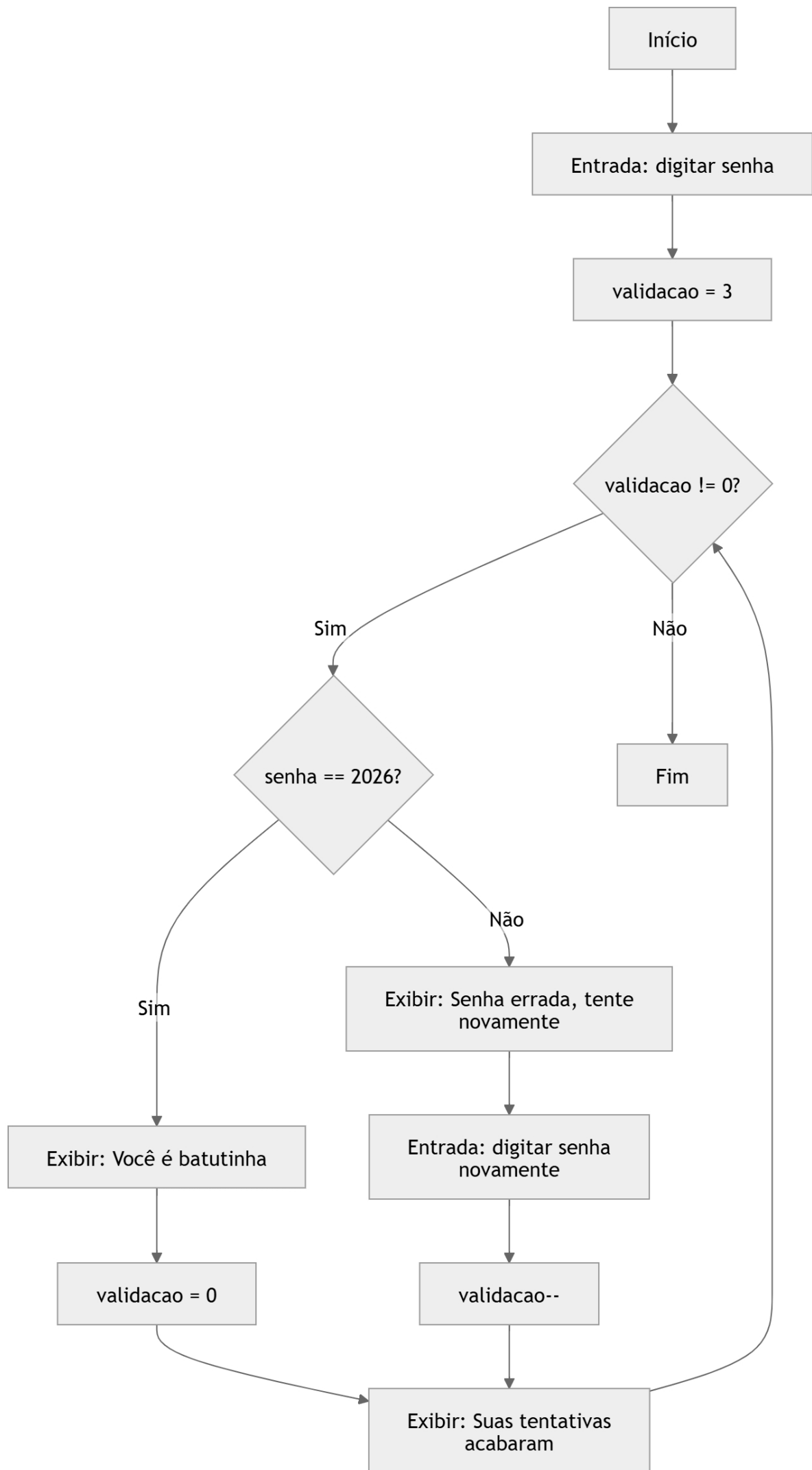
- **Nível:** Médio
- **Enunciado:** Escreva um programa que simule a validação de uma senha numérica (ex.: 2026). O programa deve permitir até 3 tentativas. Se o usuário acertar, exiba:

"Acesso Permitido!"

- Se esgotar as tentativas, exiba:

"Terminal Bloqueado!"

```
import java.util.Scanner;
public class senha{
    Run | Debug
    public static void main(String[] args){
        Scanner scanner = new Scanner(System.in);
        System.out.println(x: "Digite sua senha");
        String senha = scanner.nextLine();
        int validacao=3;
        while(validacao!=0){
            if(senha.equals(anObject: "2026")){
                System.out.println(x: "Você é batutinha");
                validacao = 0;
            }
            else{
                System.out.println(x: "Senha errada, tente novamente");
                senha = scanner.nextLine();
                validacao--;
            }
            System.out.println(x: "Suas tentativas acabaram");
        }
        scanner.close();
    }
}
```



Algoritmos e Programação – AP

▪ Exercício Proposto 6

- **Nível:** Médio
- **Enunciado:** Em uma frente de pavimentação, deseja-se saber a altura média de 10 blocos de concreto. O programa deve ler as 10 alturas, uma de cada vez, e, ao final, mostrar a média das alturas e a maior e a menor altura digitada.


```
import java.util.Scanner;

public class concreto {
    Run | Debug
    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.println(x: "Digite as 10 alturas:");

        double altura = 0;
        int vezes = 0;
        double soma = 0;

        double max = Double.MIN_VALUE;
        double min = Double.MAX_VALUE;

        while (vezes != 10) {
            altura = scanner.nextDouble();

            if (altura > max) {
                max = altura;
            }

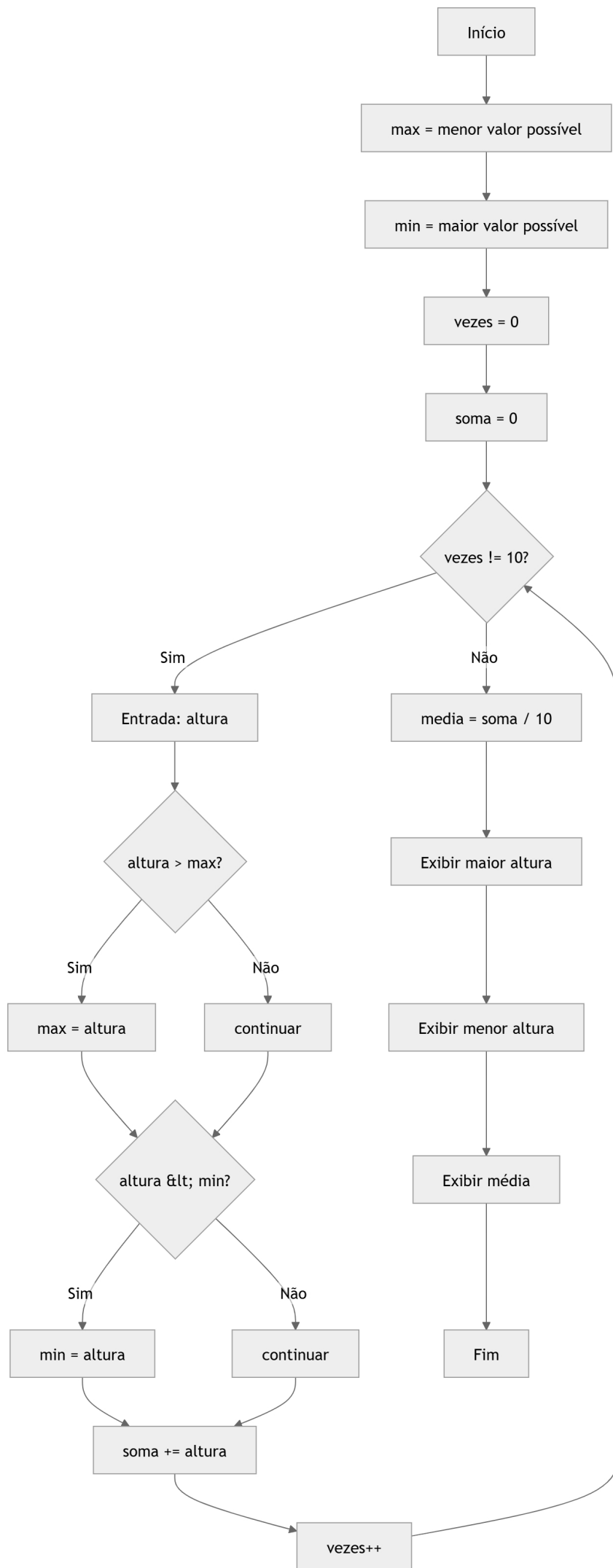
            if (altura < min) {
                min = altura;
            }

            soma += altura;
            vezes++;
        }

        System.out.printf(format: "A maior altura apresentada é %.2f\n", max);
        System.out.printf(format: "A menor altura apresentada é %.2f\n", min);

        double media = soma / 10;
        System.out.printf(format: "A média das alturas é %.2f", media);

        scanner.close();
    }
}
```



Algoritmos e Programação – AP

▪ Exercício Proposto 7

- **Nível:** Médio
- **Enunciado:** A Sequência de *Fibonacci* inicia com 0 e 1, e cada termo subsequente é a soma dos dois anteriores:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34 ...

- Peça ao usuário um número n e imprima os primeiros n termos da sequência usando um laço *while*.

```
import java.util.Scanner;

public class fibonacci {
    Run | Debug
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println(x: "Digite o numero inicial:");
        int primeiro = scanner.nextInt();

        System.out.println(x: "Digite o numero de vezes:");
        int vezes = scanner.nextInt();

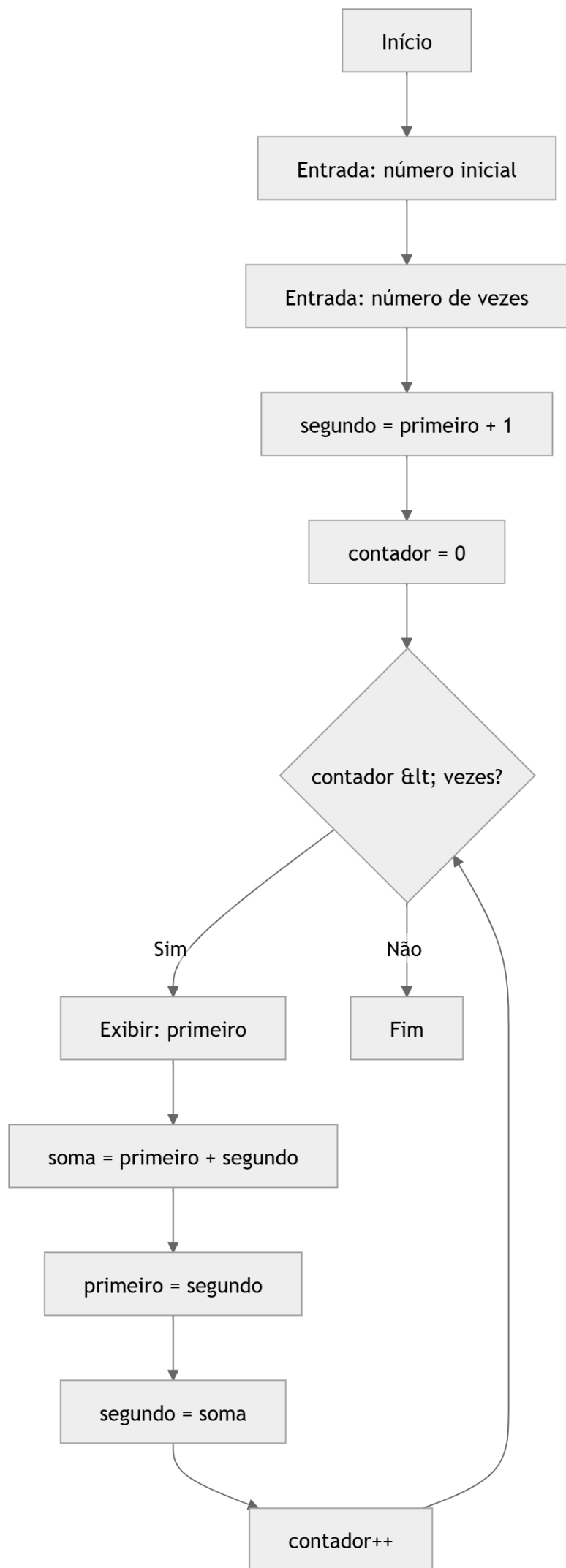
        int segundo = primeiro + 1; // forma simples de começar
        int contador = 0;

        while (contador < vezes) {
            System.out.println(primeiro);

            int soma = primeiro + segundo;
            primeiro = segundo;
            segundo = soma;

            contador = contador + 1;
        }

        scanner.close();
    }
}
```



Algoritmos e Programação – AP

▪ Exercício Proposto 8

- **Nível:** Médio
- **Enunciado:** Gerar uma tabela comparativa de graus *Kelvin* para *Celsius*. O usuário informa o valor inicial (em °K), o valor final e o passo (incremento). O programa deve exibir a tabela formatada enquanto o valor atual não ultrapassar o limite final.

```
import java.util.Scanner;

public class temperatura {
    Run | Debug
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println(x: "Digite o valor inicial em Kelvin:");
        double kelvin = sc.nextDouble();

        System.out.println(x: "Digite o valor final:");
        double valorFinal = sc.nextDouble();

        System.out.println(x: "Digite o incremento:");
        double incremento = sc.nextDouble();

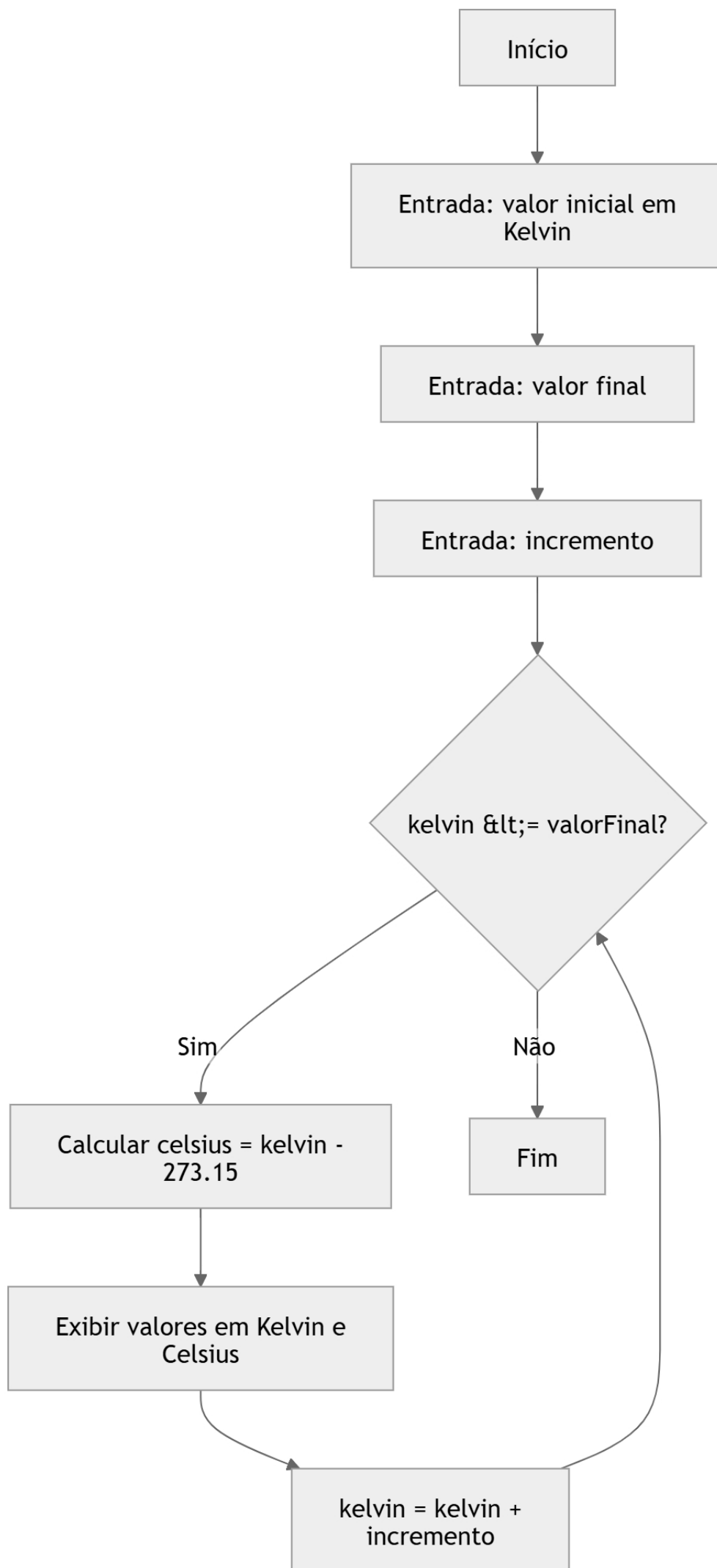
        while (kelvin <= valorFinal) {

            double celsius = kelvin - 273.15;

            System.out.printf(format: "Valor em K: %.2f = %.2f °C\n", kelvin, celsius);

            kelvin += incremento;
        }

        sc.close();
    }
}
```



Algoritmos e Programação – AP

▪ Exercício Proposto 9

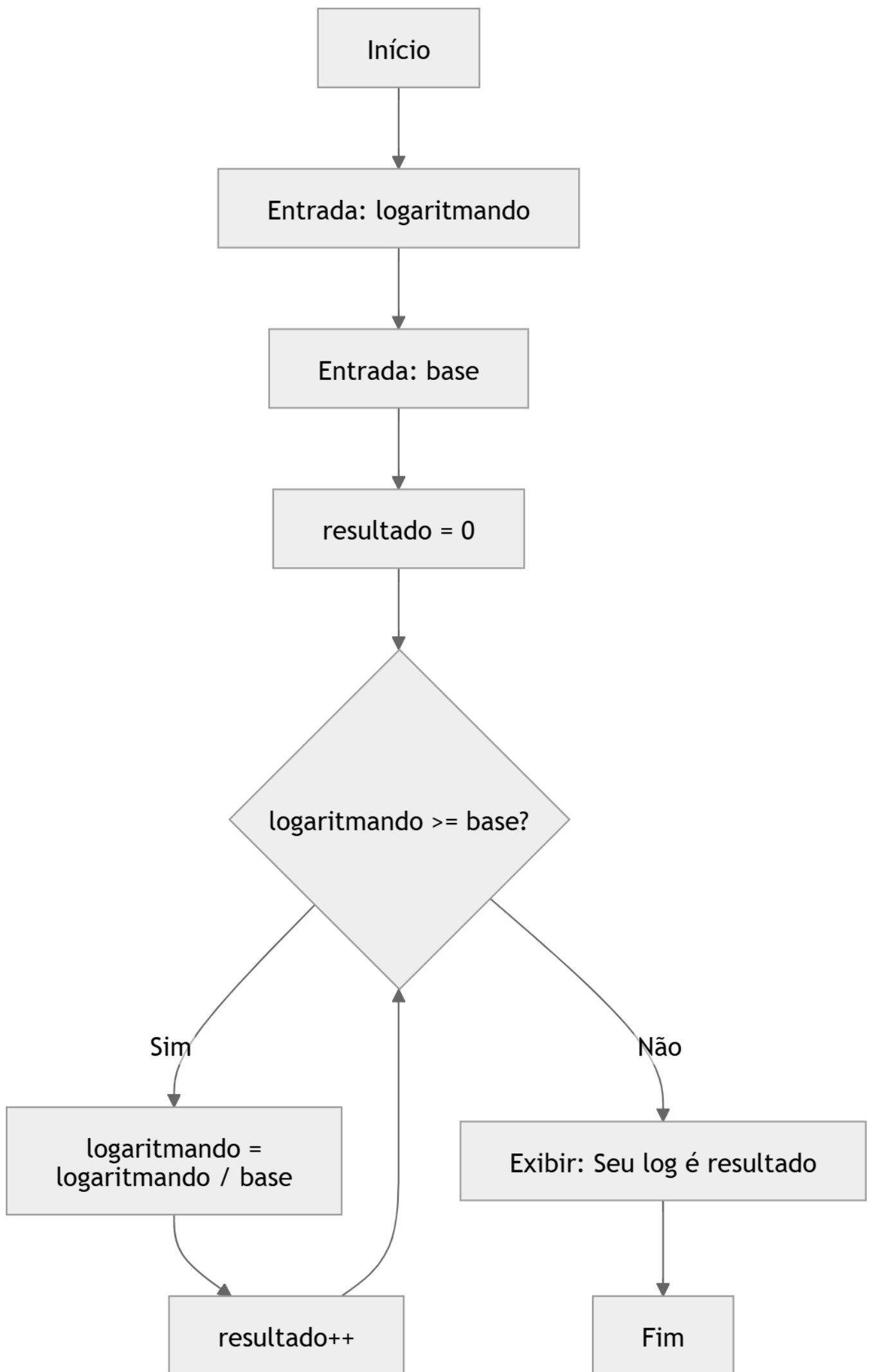
- **Nível:** Difícil
- **Enunciado:** Sem usar *Math.log()*, crie um programa que receba uma base b e um número n . Utilize um laço *while* para descobrir quantas vezes n pode ser dividido por b até chegar a um valor menor ou igual a 1 (simulando a ideia de logaritmo na base b).

```
import java.util.Scanner;

public class logaritmo{
    Run | Debug
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.println(x: "Digite o logaritmando");
        double logaritmando = sc.nextDouble();
        System.out.println(x: "Escreva a base");
        double base = sc.nextDouble();
        double resultado = 0;
        while(logaritmando >= base){
            logaritmando/=base;
            resultado ++;

        }
        System.out.println("Seu log é : "+ resultado );

        sc.close();
    }
}
```



Algoritmos e Programação – AP

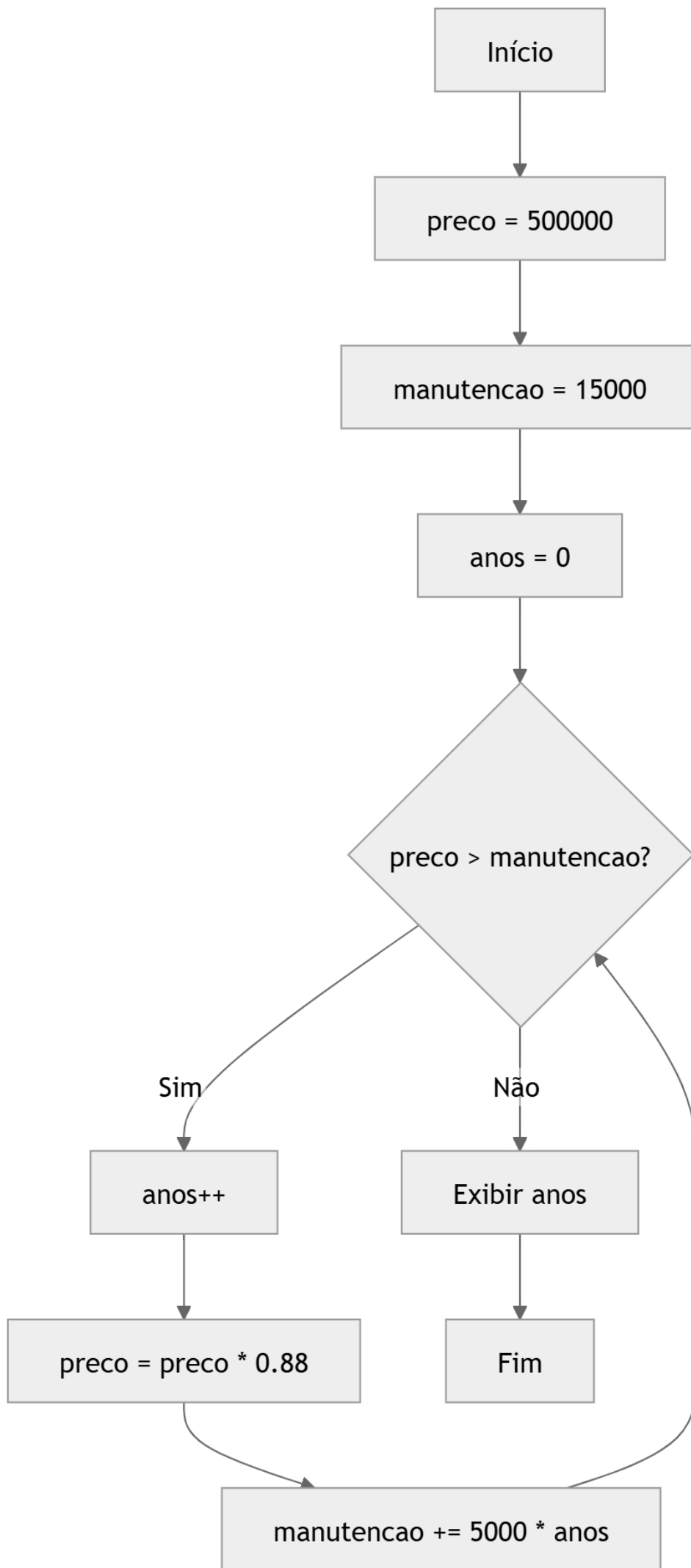
▪ Exercício Proposto 10

- **Nível:** Difícil
- **Enunciado:** Uma escavadeira custa R\$ 500.000,00. A cada ano de uso, seu valor de mercado cai 12% em relação ao ano anterior. Além disso, a manutenção anual custa R\$ 15.000,00 no primeiro ano e aumenta R\$ 5.000,00 a cada ano subsequente. Calcule em quantos anos o custo acumulado de manutenção ultrapassará o valor de mercado da máquina.

```
import java.util.Scanner;

public class escavadeira{
    Run | Debug
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        double preco = 500000;
        double manutencao = 15000;
        int anos = 0;
        while(preco > manutencao){
            anos++;
            preco*=0.88;
            manutencao += (5000 * anos);
        }
        System.out.println(anos);

        sc.close();
    }
}
```



Algoritmos e Programação – AP

▪ Exercício Proposto 11

- **Nível:** Difícil
- **Enunciado:** Leia um número inteiro positivo e determine se ele é um *número primo*. O laço *while* deve testar os divisores possíveis e interromper a execução assim que encontrar um divisor ou atingir a raiz quadrada do número (otimização).

```
import java.util.Scanner;

public class primo {
    Run | Debug
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

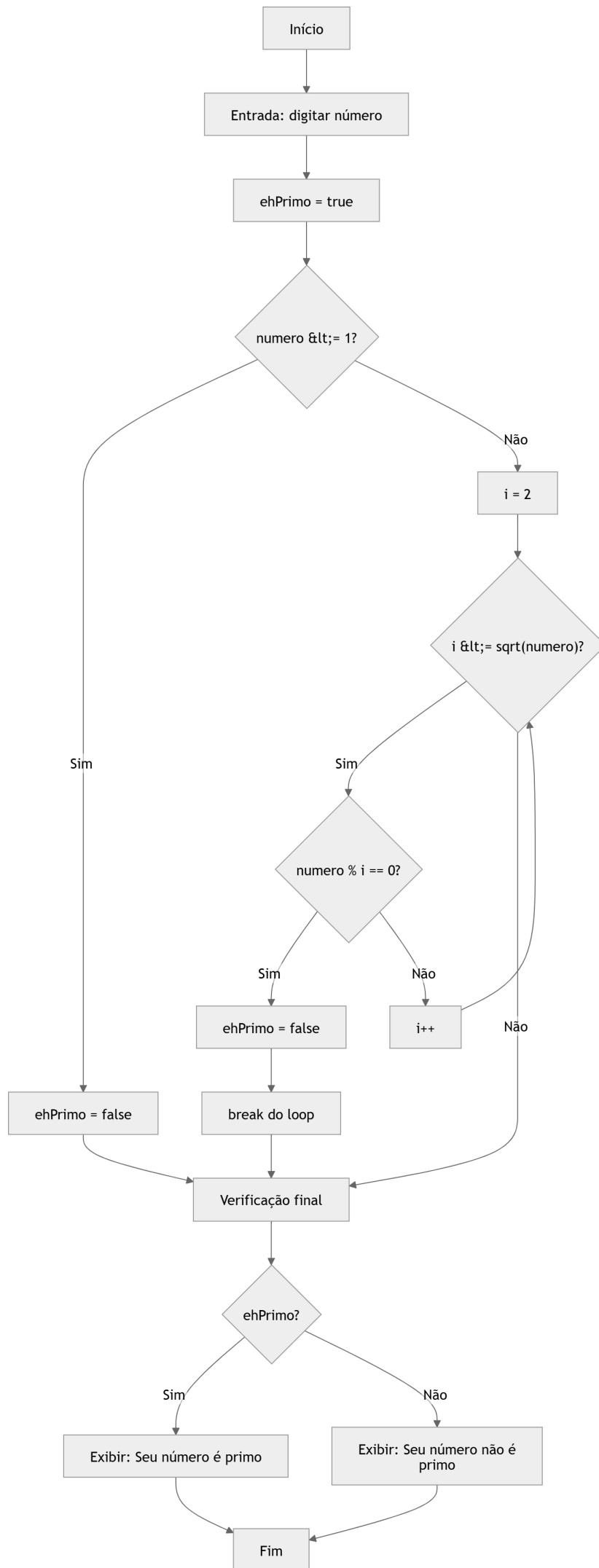
        System.out.print(s: "Digite um número: ");
        int numero = sc.nextInt();

        boolean ehPrimo = true;

        if (numero <= 1) {
            ehPrimo = false;
        } else {
            for (int i = 2; i <= Math.sqrt(numero); i++) {
                if (numero % i == 0) {
                    ehPrimo = false;
                    break;
                }
            }
        }

        if (ehPrimo) {
            System.out.println(x: "Seu número é primo");
        } else {
            System.out.println(x: "Seu número não é primo");
        }

        sc.close();
    }
}
```

Algoritmos e Programação – AP

▪ Exercício Proposto 12

- **Nível:** Difícil
- **Enunciado:** Calcular o valor por aproximação de π pela Série de *Gregory-Leibniz*:

$$\pi = 4 \times (1 - 1/3 + 1/5 - 1/7 + 1/9 - 1/11 + 1/13 \dots)$$

- Peça ao usuário o número de termos de precisão (1, 2, 3, 4 ...) e utilize o *while* para calcular o valor aproximado de π . Na tela, compare o resultado encontrado com o valor dado por ***Math.PI*** do *Java*. Rode o programa várias vezes, incrementando manualmente o número de termos de precisão digitado (1, 2, 3, 4 ...).

```
import java.util.Scanner;

public class piAproximado {
    Run | Debug
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print(s: "Digite o número de termos: ");
        int termos = sc.nextInt();

        int contador = 0;
        double soma = 0;
        double denominador = 1;
        int sinal = 1;

        while (contador < termos) {

            soma += sinal * (1 / denominador);

            denominador += 2;
            sinal *= -1;
            contador++;
        }

        double piAprox = 4 * soma;

        System.out.printf(format: "Valor aproximado de PI: %.10f\n", piAprox);
        System.out.printf(format: "Valor de Math.PI:      %.10f\n", Math.PI);

        sc.close();
    }
}
```

